0128. Longest Consecutive Sequence

ategory: Arrays & Hashing | LeetCode: https://leetcode.com/problems/longest-co

Problem

- Given an unsorted array, return the length of the longest run of consecutive integers (values, not positions), in O(n) time.
- Example: [100,4,200,1,3,2] → 4 (the run 1,2,3,4).

Intuition

- Put everything in a HashSet. A number x starts a run only if x-1 is not in the set.
- From each start, count upward x+1, x+2, ... while present.
- Each number is visited at most twice (once in the outer loop, once in some run), so the total is O(n).

Brute Force

- Sort and scan for runs (skip duplicates): O(n log n).
- Or for every x count upwards without the "start" check: O(n^2) worst case.

Optimal Approach

- Set = all values.
- For each x in set: if !set.contains(x-1), walk y = x while set.contains(y+1), y++.
- Best = max(best, y - x + 1).

Worked Example

```
nums = [100, 4, 200, 1, 3, 2].

x=100: 99 absent -> start, 101 absent -> len 1
x=4  : 3 present -> skip (not a start)
x=200: start -> len 1
x=1  : 0 absent -> start, 2,3,4 present, 5 absent -> len 4
x=3,2: not starts -> skip
best = 4
```

Java Solution

```java
public class LongestConsecutiveSequence {
    public int longestConsecutive(int[] nums) {
        Set<Integer> set = new HashSet<>();
        for (int x : nums) set.add(x);
        int best = 0;
        for (int x : set) {
            if (set.contains(x - 1)) continue;
            int y = x;
            while (set.contains(y + 1)) y++;
            best = Math.max(best, y - x + 1);
        }
        return best;
    }
}
```

Complexity

- Time: O(n) average.
- Space: O(n).

Pitfalls

- Iterate over the set, not the array, so duplicates do not repeat runs (otherwise [1,1,1,...] style inputs can degrade).
- Empty input returns 0.